LAB 4 Results

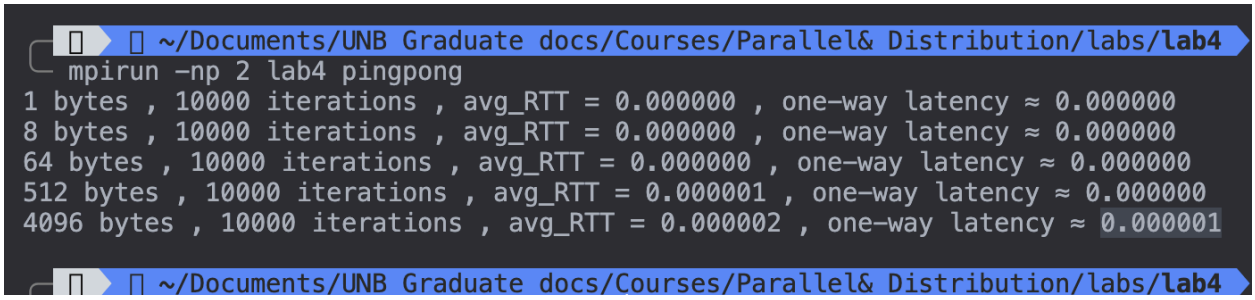
Compilation:
mpicc -O3 lab4.c -o lab4

Execution:
mpirun -np 2 lab4 pingpong
mpirun -np 6 lab4 ring

1. **Ping Pong Results**
   With the ping pong experiment we are sending message from Rank 0 to Rank1 and vice versa from Rank1 to Rank 0 . We did 10000 iteration sending messages of different bytes (1,8,64,512,4096).

   Result table:

Bytes	Iterations	Avg RTT (sec)	One way latency (sec)
1	10000	0.000000	0.000000
8	10000	0.000000	0.000000
64	10000	0.000000	0.000000
512	10000	0.000001	0.000000
4096	10000	0.000002	0.000001



Observation:
Very small messages bytes (1,8,64) appear as 0.000000 because communication is extremely fast on a single machine, larger messages slightly increase Avg RTT because more data must be transferred.

a. **How message size is controlled?**
   Message size is controlled using a raw byte buffer as its size determines the payload in this way
   char *buffer = malloc(message_size);
   MPI_Send(buffer, message_size, MPI_BYTE, destination, tag, MPI_COMM_WORLD);

   The message_size byte determines the number of bytes to be sent and the MPI_byte ensures that each element represents one byte

b. **How correctness is verified?**
   The Correctness was verified using MPI message tags rather than message contents we calculated the tag with the formula given (tag = k % TAG_RANGE) and used **status.MPI_TAG** after the receiver gets the message to check if there is a mismatch and prints it.

c. **why your implementation avoids deadlock?**
   In implementing the ping pong it starts with Rank 0 which sends data while Ranks 1 Receives data and this prevents any both the ranks to keep waiting while both are waiting to receive or send at the same time.

2. **Ring Communication Results**
   For the ring communication it connects or the processes in a ring typology

   a. **Consider this code pattern: MPI_Send(... right ...); MPI_Recv(... left ...); Question: Why can this deadlock if all processes execute it simultaneously?**
      If all the processes executed this sequence simultaneously every process would attempt to send first and as sending requires a receiver to be ready, all processes would wait indefinitely causing a deadlock.

   b. **MPI_Sendrecv used in ring Operation**
      This function performs sending and receiving in a coordinated operation, Mpi internally manages the communication so that processes do not wait indefinitely for each other which prevents the deadlock

   c. **Output results**

